B-TRUST

Microbanking & Interest Management System

Four-Week Team Execution and Commit Plan

TEAM	5 members	**DURATION**	4 weeks
BASE	B-Trust MIMS Next.js starter	**FOCUS**	Database backend + lightweight QA UI

Purpose

This is the team's operational plan: who owns each feature, which commits should be produced, when work is reviewed, and what counts as complete. Replace Member 1-5 with your names at the first meeting.

Version 1.0 | Prepared 16 September 2026

1. Project outcome and scope

At the end of four weeks, the team must deliver a working database-backed MIMS that satisfies the official brief and can be explained confidently in the viva.

Required functional scope

- Branches, agents and customer registration, including customer-to-agent assignment.
- Individual and joint savings accounts with Children, Teen, Adult, Senior and Joint plans.
- Deposits, withdrawals, minimum-balance enforcement, overdraft prevention and full transaction logging.
- Fixed deposits for 6 months, 1 year and 3 years; one FD per savings account.
- Monthly FD interest processing every 30 days, credited as a transaction to the linked savings account.
- Five management reports required by the brief.
- Sample data: at least 3 branches, 5 agents, 15 customers, 2 joint accounts, 10 FDs and 100 transactions.

Not in scope

- Loans, cards, messaging, email verification, mobile apps and complex animations.
- A production-grade banking security deployment. The goal is a clear academic database project with a lightweight QA interface.

Completion rule

A feature is complete only when its SQL works, the API and UI use it, valid and invalid cases are tested, the owner can explain it, and the change is reviewed and merged into develop.

2. Member ownership

Each member owns one vertical slice from database to screen. This makes contribution visible and avoids one person becoming the only member who understands SQL.

Member	Primary feature ownership	Core database objects	Secondary team duty
Member 1 Integration lead	Branches, agents, users, login and role access	branches, agents, users, roles; login query	Repo health, integration builds, environment guide
Member 2 Customer lead	Customer registration and profile operations	customers; validation and lookup queries	Assumptions log and ERD coordination
Member 3 Account lead	Savings plans, individual/joint account opening	savings_plans, savings_accounts, account_holders	Schema consistency and constraints review
Member 4 Transaction lead	Deposit, withdrawal and account history	transactions; deposit/withdraw procedures	ACID, concurrency and negative tests
Member 5 FD/report lead	Fixed deposits, interest processing and reports	fd_plans, fixed_deposits, interest_distributions, report views/queries	Seed-data totals and report validation

Shared knowledge requirements

- All five members approve the ERD, keys, relationships and business assumptions before schema implementation.
- All five members review the deposit and withdrawal procedures because ACID and concurrency are likely viva topics.
- Every member reviews at least one pull request outside their own feature.
- During Week 4, each member demonstrates one feature they did not originally own.

Naming

At the kickoff meeting, record: Member 1 = _____ Member 2 = _____ Member 3 = _____ Member 4 = _____ Member 5 = _____

3. Git and commit workflow

Uploading the complete extracted starter project in one initial commit is acceptable. That commit is only the baseline. After it, members must use focused branches and small commits so contribution and reasoning are visible.

Branches

Branch	Use	Rule
main	Stable milestones only	No direct coding; merge from develop after a milestone passes
develop	Integrated team version	All feature pull requests target this branch
feature/m1-auth	Member 1 work	Keep updated from develop before review
feature/m2-customers	Member 2 work	One logical change per commit
feature/m3-accounts	Member 3 work	Do not mix unrelated formatting
feature/m4-transactions	Member 4 work	Include rollback and failure tests
feature/m5-fd-reports	Member 5 work	Separate FD and report commits where possible

Commit rules

- Use: type: short action. Types: feat, fix, test, docs, refactor, chore.
- A commit should run and represent one understandable change. Do not wait until Friday to push one huge commit.
- Never commit node_modules, .next, .env, .env.local, passwords or database dumps containing private data.
- Every pull request states: what changed, how to test, affected SQL objects, screenshots if UI changed, and known limitations.
- Reviewer checks the code and asks the owner to explain the important query before approval.

Initial baseline commit

Member 1 creates the repository and commits the extracted starter as: chore: add MIMS starter project. Then create develop and protect main from direct edits.

4. Week 1 - Design and database foundation

Goal: one agreed model, a runnable schema and controlled sample foundation. No member should independently invent conflicting tables.

Owner	Planned commits	Evidence / reviewer
M1	chore: configure local environment example feat: create roles users branches and agents tables feat: seed branches agents and test users	Fresh setup works; reviewed by M3
M2	docs: record domain assumptions and customer rules feat: create customers table and constraints feat: add customer validation seed data	Age/contact rules tested; reviewed by M5
M3	docs: add agreed ERD and relational mapping feat: create savings plans accounts and holders tables feat: seed account plans	Individual/joint relationships tested; reviewed by M4
M4	docs: define transaction states and business-hour rule feat: create transaction table and reference constraints test: add schema integrity test cases	Duplicate reference and invalid FK rejected; reviewed by M1
M5	feat: create fd plans deposits and interest tables feat: seed fixed deposit plans docs: add report data requirements	One-FD-per-account rule designed; reviewed by M2

Team checkpoints

When	Checkpoint	Exit condition
Day 1	Kickoff and requirements walkthrough	Names assigned; assumptions list opened; starter runs for all
Day 2	ERD workshop	Entities, cardinalities, PKs and FKs agreed
Day 3	Schema review	Data types, CHECK/UNIQUE constraints and deletion rules agreed
Day 5	Foundation milestone	A clean database can run schema.sql then seed.sql without errors

Week 1 deliverable

Merge a tagged milestone v0.1-schema: assumptions, ERD, normalized relational schema, initial constraints and seed plans. Do not move unfinished/conflicting SQL into Week 2.

5. Week 2 - Core banking features

Goal: customer and savings-account flows work end-to-end; deposits and withdrawals are atomic and correctly logged.

Owner	Planned commits	Done when / reviewer
M1	feat: implement database-backed login query feat: enforce role access in middleware test: cover invalid login and forbidden role	Admin/agent/customer access verified; M4 reviews
M2	feat: implement customer CRUD API feat: connect customer registration form test: cover duplicate and invalid customer data	New customer persists and is assigned to agent; M5 reviews
M3	feat: implement account opening API feat: connect individual and joint account forms test: enforce plan eligibility and holder rules	Valid accounts open; invalid age/joint cases fail; M2 reviews
M4	feat: implement atomic deposit procedure feat: implement safe withdrawal procedure feat: connect transaction API and form test: cover overdraft rollback and minimum balance	Balance and transaction log change together; M1 + whole team review
M5	feat: implement account transaction history query feat: connect transaction history page test: verify totals and ordering	History matches database and shows references/timestamps; M3 reviews

Mandatory transaction review

- Use START TRANSACTION, SELECT ... FOR UPDATE, COMMIT and an error handler that ROLLBACKs.
- Reject zero/negative amounts, inactive accounts, overdrafts and withdrawals below plan minimum.
- Generate or validate a unique reference number and log the acting agent/customer where applicable.
- Run two competing withdrawal tests and confirm the final balance remains valid.

Week 2 deliverable

Merge and tag v0.2-core-banking. Demonstrate: register customer -> open account -> deposit -> withdraw -> view history, including one rejected withdrawal.

6. Week 3 - Fixed deposits, reports and full data

Goal: finish the distinctive database features and produce the complete required dataset.

Owner	Planned commits	Done when / reviewer
M1	feat: add admin process-interest endpoint feat: add dashboard counts test: restrict central interest action to admin	Role checks and error responses verified; M4 reviews
M2	feat: add customer activity report query and UI test: validate deposits withdrawals and net totals chore: expand customers to required count	15 customers and correct per-customer totals; M5 reviews
M3	feat: add account summary report query and UI test: reconcile calculated activity with current balance chore: add joint account seed cases	At least 2 valid joint accounts; M2 reviews
M4	feat: add agent transaction report query and UI test: verify transaction counts and values by agent chore: generate mixed transaction seed data	100+ mixed transactions; report totals reconcile; M1 reviews
M5	feat: implement fixed deposit creation feat: implement monthly interest procedure feat: add active FD and interest reports test: prevent second FD and duplicate payout	10 FDs; next dates correct; interest creates one credit transaction; M3 reviews

Report acceptance checks

#	Required report	Validation
1	Agent-wise transaction count and value	Manual sample sum equals output
2	Account-wise summary and current balance	Reported current balance equals account table
3	Active FDs and next payout dates	Only active FDs; date is last/initial payout + 30 days
4	Monthly interest by account type	Month filter and grouped total verified
5	Customer deposits, withdrawals and net	Joint-account attribution assumption documented

Week 3 deliverable

Merge and tag v0.3-complete-features. All five reports run, FD interest is idempotent for a payout cycle, and minimum sample-data counts are proven with count queries.

7. Week 4 - Integration, testing and viva

Goal: stop adding features, remove defects, prove correctness and prepare every member to explain the full system.

Owner	Planned commits	Final responsibility
M1	fix: resolve integration and authorization defects docs: finalize setup and run instructions	Fresh-machine setup and final merge coordination
M2	test: complete customer and validation test matrix docs: document assumptions and normalization	ERD/schema section of report
M3	test: complete account and joint-holder cases docs: document keys constraints and indexes	Data model and integrity viva section
M4	test: complete ACID concurrency and rollback tests docs: explain transaction procedures	Live deposit/withdraw demonstration
M5	test: reconcile FD interest and all report totals docs: document procedures functions triggers reports	FD/report demonstration and sample-data proof

Four-day freeze sequence

Day	Focus	Rule
Day 1	End-to-end integration	Run every core flow from a clean database
Day 2	Negative and boundary testing	Fix only reproducible defects; record test evidence
Day 3	Report, screenshots and demo rehearsal	No schema changes unless a critical defect exists
Day 4	Viva rotation and final release	Each member explains one owned and one unowned feature

Release gate

Merge develop into main and tag v1.0-final only after a clean setup, all test cases pass, data counts meet the brief, every report reconciles, and no secrets or generated folders are tracked.

8. Review, testing and definition of done

Use this checklist for every pull request. A feature is not finished merely because the page opens.

Area	Required check
Database	PK/FK/UNIQUE/CHECK constraints are appropriate; money uses DECIMAL; query runs on a fresh schema.
Business rules	Eligibility, minimum balance, active status, business hours, one-FD limit and no-overdraft rules are enforced at the correct layer.
ACID	Related balance and history writes occur in one transaction; errors roll back; concurrent updates are protected.
API	Input is validated; correct status and useful message returned; SQL parameters are bound rather than concatenated.
UI	Success, empty, loading and error states are understandable; forms prevent obvious invalid input.
Tests	At least one normal, boundary and invalid case recorded with expected and actual results.
Knowledge	Owner can explain tables, joins, procedure flow, failure behavior and test evidence without reading generated text.
Review	A teammate has reviewed it and the integrated develop branch still runs.

Critical system test sequence

- Create branch and agent; register eligible customer; open individual account.
- Open a joint account with at least two holders; verify both can transact under the chosen assumption.
- Deposit, withdraw valid amount, reject overdraft, reject minimum-balance violation, then confirm history.
- Open FD, reject second active FD on the same account, process a due interest payout once, reject duplicate payout.
- Run all five reports and reconcile several outputs manually against raw SQL totals.
- Rebuild from zero using documented commands and run the same smoke test.

Daily 15-minute stand-up

Each member answers: What merged yesterday? What exact commit will I finish today? What am I blocked by? Which pull request will I review? Is develop currently runnable?

9. Team calendar and viva ownership

Week	Mon-Tue	Wed-Thu	Fri checkpoint
1	Requirements, assumptions, ERD	Schema objects and seeds	v0.1 schema demo
2	Auth, customer and account flows	Deposit, withdrawal and history	v0.2 banking demo
3	FD and interest processing	Reports and full sample data	v0.3 feature demo
4	Integration and negative tests	Documentation and rehearsals	v1.0 final release

Minimum viva preparation

Member	Must lead	Must also explain
M1	Authentication, roles, branch/agent model	Deposit transaction lifecycle
M2	Customer model, assumptions, normalization	Joint account mapping
M3	Plans, accounts, holders, constraints	One management report query
M4	ACID, locking, rollback, concurrency	FD interest lifecycle
M5	FDs, 30-day payout, reports/indexes	Customer and account creation flow

Questions everyone must answer

- Why is account_holders needed instead of storing customer_id directly in savings_accounts?
- Where are minimum balance and eligibility rules enforced, and why?
- How does SELECT ... FOR UPDATE prevent two withdrawals from overspending the same balance?
- Why is DECIMAL used for money? What would go wrong with FLOAT?
- How do you prevent duplicate monthly interest credits?
- Which indexes support transaction history, next payout lookup and report grouping?
- What happens step by step when any procedure fails halfway?

Final principle

AI-assisted code is acceptable only when the owner has tested it and can explain every table, join, condition, transaction and failure case. Understanding is part of the deliverable.